

Understanding smolVLA: Training and Evaluation Details

Finetuning, Losses, and the Overfitting Paradigm

1 Introduction

Training a **smolVLA** model involves fine-tuning a pretrained Vision-Language-Action model on a specific robot dataset. This process differs from standard computer vision or NLP training in how it measures success and how it handles data validation.

This document clarifies the interpretation of training metrics (the Loss) and the specific evaluation strategy used in the LeRobot framework.

2 The Pre-Training Startup Sequence

Before the training logs start appearing, `lerobot-train` performs a sequence of initializations visible in the trace:

1. **Config Summary:** A JSON-like block showing every parameter (batch size, learning rate, etc.). It confirms the **Effective Batch Size** (in your trace: $8 \times 1 = 8$).
2. **Dataset Creation:** Reports `num_frames` (16,914) and `num_episodes` (33). This is where the foundation of the epoch math is laid.
3. **Policy Initialization:** Confirms the model architecture (`smolvla`) and whether weights are loaded from a checkpoint. For VLM-based models, it also handles reduction of layers (e.g., reducing to 16 layers).
4. **Optimizer & Scheduler Scaling:** An important log entry: *"Auto-scaling LR scheduler"*. Since the actual steps (26,000) were less than the default decay steps (30,000), the scheduler automatically compressed the warmup and decay phases to fit the requested limit.

3 Interpreting the Training Loss

During training, the primary metric displayed in the logs (e.g., via `wandb` or terminal output) is the **Training Loss**. As established in the Flow Matching framework, this loss is the **Mean Squared Error (MSE)** between the target velocity vector u_τ and the model's predicted velocity v_τ .

3.1 What does a loss value mean?

Let's analyze a typical loss value, such as $\mathcal{L} = 0.02$.

1. **The Mathematical Definition:** The loss is the average of squared differences across all elements of the action chunk. For a standard smolVLA configuration (50 timesteps $\times$ 32 joints), each loss value is an average over 1,600 individual predictions.

$$\text{MSE} = \frac{1}{N} \sum (v_{i,j} - u_{i,j})^2$$

2. **Root Mean Squared Error (RMSE):** To understand the "average error" in the same units as the velocity, we take the square root of the loss:

$$\text{RMSE} = \sqrt{0.02} \approx 0.1414$$

3. **Scale Comparison:**

- The target velocity u_τ is defined as Noise – Action.
 - Since the **Noise** is sampled from a standard Gaussian $\mathcal{N}(0,1)$, its values typically range from -3 to 3 .
 - The **Actions** are normalized by the LeRobot dataset statistics, typically mapping them to the $[-1,1]$ range.
 - Therefore, the target u_τ often spans a range of approximately -4 to 4 .
4. **Intuition:** If your loss is 0.02, it means that on average, the model’s prediction for each of the 1,600 velocity components is off by only ≈ 0.14 . Given that the target values can be as large as 3 or 4, an error of 0.14 represents a very high degree of precision in predicting the flow field.

4 The Question of Overfitting and Validation

In standard machine learning, we carefully split data into *Training*, *Validation*, and *Test* sets to detect overfitting. However, the `lerobot-train` pipeline for offline imitation learning often behaves differently.

4.1 Is there an Offline Validation Set?

By default, the `lerobot_train.py` script **does not perform an offline train/test split** of the demonstration data to compute a "Validation Loss."

The training loop cycles through the entire provided dataset continuously. There are several reasons for this design choice:

1. **Data Scarcity:** Robot demonstration datasets are often small (50 to 100 episodes). Splitting off 20% of this precious expert data for a validation loss would significantly reduce the diversity of behaviors the model can learn.
2. **Loss vs. Success:** In Robotics, **Offline Loss is a poor proxy for Online Success.** A model can have a very low validation loss (it predicts the expert’s next move well on paper) but fail miserably in the real world because it hasn’t learned to recover from small deviations (the "compounding error" problem).

4.2 If there’s no Val Set, how do we evaluate?

Evaluation in LeRobot is performed **"In-the-Loop"** rather than **"On-Paper"**:

- **Environmental Evaluation** (`eval_freq`): Every N steps (e.g., 20,000), the trainer pauses and runs the current policy in a **Simulator** (e.g., Gym, Mujoco, or Isaac Gym).
- **True Metrics:** Instead of looking at MSE, we look at the **Success Rate** (percentage of tasks completed) and **Average Reward**. This is the only "validation" that truly matters for a robot.
- **Real-World Limitation:** For real-world datasets where no simulator exists, there is indeed no automatic validation. Practitioners usually train for a fixed number of steps, then manually test the checkpoints on the physical robot.

4.3 Is Overfitting a Problem?

In Imitation Learning (Behavioral Cloning), **overfitting is often precisely the goal**. We want the model to mimic the expert as perfectly as possible.

The "Flow Matching" framework naturally mitigates the traditional dangers of overfitting:

- By sampling new random noise matrices for every single batch, we are effectively performing massive **Data Augmentation**. The model never sees the exact same "noisy chunk" twice, which forces it to learn the underlying **Vector Field** rather than memorizing specific sequences.
- Even if the model "overfits" the training demonstrations, its ability to denoise starting from **Random Noise** at inference time provides it with inherent robustness and stochastic variety.

5 Action Scaling: From Degrees to Latents

A natural question arises: if robot sensors report angles from -180° to 180° , how do we add noise to those large numbers?

The answer is that the actions are **Normalized** before the training loop ever sees them.

1. **Statistics Collection:** Before training starts, LeRobot calculates the **mean** and **std** (standard deviation) for every single joint and motor across the entire dataset.
2. **The Transform:** During training, the raw actions are transformed:

$$\text{Action}_{\text{norm}} = \frac{\text{Action}_{\text{raw}} - \text{mean}}{\text{std}}$$

3. **Stable Noising:** This maps the actions to a range roughly around $[-1, 1]$. Now, when we add Gaussian noise ($\mathcal{N}(0, 1)$), the noise is of a similar magnitude to the signal. This ensures that the model can actually "see" the noise it needs to remove.

6 The Simulator: Beyond the MSE

You asked if the simulator just measures MSE on noised chunks. The answer is **No**—that would be "Offline Evaluation." The simulator evaluation is much more powerful: it is a **Rollout**.

6.1 How the Simulator Rollout Works

When `lerobot-train` pauses for evaluation, it doesn't just look at expert data. It lets the model "drive" the robot in a virtual environment:

1. **Reset:** The simulator (e.g., Mujoco or Gym) resets to a fresh starting state.
2. **The Selection Loop:**
 - The model receives a *live* image and state from the simulator.
 - The model generates a new action chunk using the full iterative Flow Matching process (10 steps of Euler integration).
 - The *first action* of that chunk is applied to the simulator's motors.
 - The simulator physics engine calculates the new state (the objects move, the arm stays or falls).
3. **Completion:** This continues until the episode ends or the robot times out.

6.2 Success vs. Loss

The simulator measures two key metrics that MSE cannot capture:

- **Success Rate:** It checks the actual outcome (e.g., "Is the block inside the red bowl?").
- **Compounding Errors:** In a rollout, if the model makes a small mistake at Step 10, the robot ends up in a position it never saw in the training data. The rollout tests if the model is **robust** enough to recover and still finish the task.

6.3 Optional Evaluation: Skipping the Simulator

It is important to note that the Simulator Evaluation is **optional**. If the configuration setting `env` is set to `null` (as it was in the `smolvla_red_block_finetune` run), the trainer will simply skip the rollout phase.

In such cases, the user relies entirely on the **Training Loss** to monitor progress. This is common when:

- No matching virtual simulator exists for the specific physical setup.
- The user wants to speed up training by avoiding the overhead of running episodes.
- The goal is to first ensure the model can converge on the expert manifold before testing live behaviors.

7 Anatomy of the Training Logs

When running `lerobot-train`, the terminal outputs a recurring line of metrics. Understanding these is key to monitoring the health of the finetuning process.

```
step:25K  smpl:202K  ep:393  epch:11.92  loss:0.010  grdn:0.222  lr:2.8e-06  updt_s:1.233
data_s:0.011
```

7.1 The Math of Epochs and Steps

A frequent point of confusion is how `step`, `ep`, and `epch` relate. Using the values from your trace:

- **The Frame Count math:** You calculated $\approx 16,830$ frames ($33 \text{ episodes} \times 17\text{s} \times 30\text{Hz}$). The reported 16,914 is slightly higher (a difference of only 84 frames). This is normal: it simply means some episodes were slightly longer than 17.0s (e.g., 17.1s). This small variation is typical in real-world recorded data.
- **Why is an Epoch $\approx 2,100$ steps?** An epoch is one full pass through the dataset. With 16,914 frames and a batch size of 8, the math is:

$$\text{Steps per Epoch} = \frac{16,914 \text{ frames}}{8 \text{ frames/step}} \approx 2,114 \text{ steps (confirming your observation)}$$

- **The Episode Count discrepancy:** If you counted 25 episodes in your local `debug_data/file-000.csv` but the trainer says 33, it is because the dataset is split into **two chunks** on the Hugging Face hub: `file-000.parquet` and `file-001.parquet`. Your local inspection script only downloaded the first chunk (episodes 0–24). The `lerobot-train` tool automatically loads all available files, totaling 33 episodes (indices 0–32).

$$\text{Epoch} = \frac{393 \text{ episodes total}}{33 \text{ episodes/dataset}} \approx 11.9 \text{ epochs}$$

This matches the log's `epch:11.92`.

7.2 Performance and Health Metrics

- **Gradient Norm (grdn):** You observed a stable 0.222. This is the L2 norm of the gradient vector for the $\approx 100M$ learnable parameters. Since the training clip value is set to 10.0, a value of 0.222 is very "quiet." It indicates the model is converging smoothly on the target flow field ($u_t = \text{noise} - \text{action}$) without any unstable "jumps" in the weight space.
- **Learning Rate (lr):** LeRobot uses a **Cosine Decay with Warmup** scheduler.
 - **Phase 1: Warmup** ($0 \rightarrow 1,000$ steps): LR rises from $0 \rightarrow 10^{-4}$.
 - **Phase 2: Cosine Decay** ($1,000 \rightarrow 26,000$ steps): LR follows a cosine curve down to the "decay" value of 2.5×10^{-6} .

In your trace at step 25K, the LR is $2.8\text{e-}06$, confirming you are at the very final refinement stage of the training.

- **Update Time (updt_s):** Why care about forward+backward time? Because it dictates your total training time. At 1.2s/step, training 26K steps takes ≈ 8.6 hours. If this duration suddenly increases, it might indicate GPU overheating or memory swapping.
- **Data Loading (data_s):** This is the time to fetch frames and construct the "prefix" (image/state embeddings). If this is low (0.01s), your CPU is successfully keeping the GPU fed with data.

8 Case Study: Interpreting the 0.08 to 0.01 Transition

A common observation in **smolVLA** finetuning is a rapid initial drop in loss, followed by a long tail of refinement. Let's analyze a session where the loss starts at 0.08 (around step 250) and converges to 0.01 (around step 25,000).

8.1 Stage 1: The Coarse Alignment ($Loss \approx 0.08$)

At the start of finetuning, the pretrained "frozen" VLM has a general understanding of images, but the "Expert" layers have not yet aligned the random Gaussian noise with the specific motor commands of the new robot.

- **Error Magnitude:** $RMSE = \sqrt{0.08} \approx 0.28$.
- **Result:** In the range of $[-4, 4]$, an error of 0.28 is significant. The model understands the *direction* of movement but produces "jittery" or noisy trajectories.

8.2 Physical Translation: From MSE to Degrees

A loss of 0.01 feels abstract. To understand it, we can derive the physical error in degrees ($^\circ$) by reversing the normalization steps.

The Derivation

Let D be the raw robot data (in degrees) and S be the scaled data (normalized to $\sigma = 1.0$). The transformation is roughly $D = s \cdot S$, where s is the dataset standard deviation.

1. **Scaled Error:** The training metric is $MSE = E[(S - P_S)^2] = 0.01$.
2. **Physical Error:** When we scale back to raw units ($P_D = s \cdot P_S$), the error becomes:

$$E[(D - P_D)^2] = E[(sS - sP_S)^2] = s^2 \cdot E[(S - P_S)^2] = s^2 \cdot 0.01$$

3. **Root Mean Squared Error (RMSE):** Taking the square root gives us the average physical error per dimension:

$$\text{RMSE}_{raw} = \sqrt{s^2 \cdot 0.01} = s \cdot 0.1$$

4. **Numerical Result:** With your dataset's average $\sigma \approx 30.5^\circ$:

$$\text{Error} \approx 30.5 \times 0.1 = \mathbf{3.05^\circ}$$

Physical Example

For a 6 DoF robot, a loss of 0.01 means that for every timestep in your 50-step chunk, the prediction is remarkably close to the expert.

Imagine a single action frame:

- **Expert (Degrees):** [15, 20, 30, 42, 10, 5]
- **Model Prediction:** [18, 17, 33, 39, 13, 8] (*Example error of $\approx 3^\circ$ per joint*)

At 0.08 loss, the error was $\approx 8.5^\circ$ per joint (noticeably "shaky"). At 0.01, the error is only 3° . For high-dimensional control, achieving a 3° variance starting from random noise is the hallmark of a successfully converged policy.

The Geometric Error Budget

Let's analyze the physical consequence for your specific robot arm (30 cm length) and the goal for future training.

- **Joint Configuration:** 3 sensors for the main arm joints, 3 for the gripper.
- **Average Error:** If each joint has an error $E[X_i] \approx 3^\circ$ ($\pi/60$ rad).
- **Total Vector Magnitude:** Assuming independent errors, the magnitude of the angular error vector is:

$$\|\mathbf{e}\| \approx \frac{\pi}{60} \cdot \sqrt{3} \text{ rad}$$

- **End-Effector Deviation:** At a lever arm of $L = 30$ cm, the average positional error at the tip is roughly:

$$\text{Error}_{tip} \approx L \cdot \frac{\pi}{60} \cdot \frac{\sqrt{3}}{2} \approx \mathbf{1.36 \text{ cm}} \text{ (approaching 1.5 cm)}$$

- **The Ideal Goal ($MSE = 0.001$):** A 1.5 cm deviation is significant for a small gripper. To achieve true precision (cutting the error by 3 down to 1°), the required Loss is:

$$\text{Target MSE} \approx \left(\frac{1}{3} \cdot \text{Current RMSE}\right)^2 \approx \left(\frac{0.1}{3}\right)^2 \approx \mathbf{0.0011}$$

Achieving a loss of **0.001** would correspond to a sub-centimeter tip precision, allowing for extremely delicate manipulation.

9 Summary

- **Normalization:** We scale the $-180/180$ range down to $\approx [-1, 1]$ before adding noise so the math stays stable.
- **Simulator:** It's an "Online" test. The robot actually tries to do the task. This is the ultimate "Validation" of the model's intelligence.
- **Flexibility:** If no simulator is available (e.g., `env: null`), the training continues as a pure offline process, evaluated purely via training loss.